Restaurant Rush: Stage 1 class diagram (CE/CZ2002 notation)

entity
«entity» Staff
#name: String
+perform(a: StaffAction, r: Restaurant, turn: int): ActionResult
+getName(): String
CustomerState
WAITING
SEATED
ORDERED
READY
SERVED
EATING
READY_TO_PAY
PAID
CANCELLED
PREPARING
OrderStatus
PLACED
PREPARING
READY
PAID
CANCELLED
ActionType
SEAT
TAKE_ORDER
SERVE
ADVANCE
WAIT
«entity» Table
-id: int
-occupant: Customer
+isFree(): boolean
+occupy(c: Customer)
+release()
«entity» Cashier
+collectPayment(r: Restaurant, turn: int): ActionResult
«entity» Waiter
+perform(a: StaffAction, r: Restaurant, turn: int): ActionResult
+takeOrder(c: Customer, r: Restaurant, turn: int): ActionResult
+serve(c: Customer, o: Order, turn: int): ActionResult
«entity» Chef
+perform(a: StaffAction, r: Restaurant, turn: int): ActionResult
«entity» StaffAction
«entity» ActionResult
«entity» Customer
-idCounter: int
-id: int
-satisfaction: int = 100
-arrivalTurn: int
-state: CustomerState
-order: Order
+lossPerTurn(): int
+discount(priceCents: int): int
+chooseItem(menu: List<MenuItem>): MenuItem
+loseSatisfaction()
+addSatisfaction(amount: int)
+cancel()
+hasLeft(): boolean
+hasLeftOrPaid(): boolean
«entity» RegularCustomer
-random: Random
«entity» Order
-item: MenuItem
-lockedPriceCents: int
-unitsDone: int = 0
-readyOnTurn: int = -1
-status: OrderStatus
+advance(turn: int): boolean
+markServed()
+markPaid()
+cancel()
+getStatus(): OrderStatus
«entity» MenuItem
-name: String
-priceCents: int
-prepUnits: int
+getName(): String
+getPriceCents(): int
+getPrepUnits(): int
2
2
1
occupant
0..1
«use»
0..*
queue / activeCustomers
«use»
staff (Waiter, Chef)
TABLE_COUNT
«use»
«use»
1
order
0..1
1
item
1
3
menu
control
«control» GameController
-currentTurn: int
-restaurant: Restaurant
-config: GameConfig
-ui: ConsoleUI
-evaluator: ResultEvaluator
+run()
«create & use»
1
«control» GameConfig
+MAX_TURNS: int = 18
+TABLE_COUNT: int = 2
+MIN_PAID: int = 4
+MIN_REVENUE_CENTS: int = 4500
+MIN_AVG_SATISFACTION: int = 60
«control» TurnProcessor
-restaurant: Restaurant
-ui: ConsoleUI
+runTurn(turn: int)
-processArrivals(turn: int)
-processEatingAndPayment(turn: int)
-applyWaitingLoss(turn: int)
1
«control» ResultEvaluator
+evaluate(r: Restaurant): Outcome
«use»
«use»
«use»
1 1
«control» Restaurant
-revenueCents: int = 0
+enqueue(c: Customer)
+findCustomerInQueue(id: int): Customer
+findFreeTable(): Table
+seat(c: Customer, t: Table): ActionResult
+findSeatedUnorderedCustomer(id: int): Customer
+findCustomerWithReadyOrder(id: int): Customer
+getPreparingOrders(): List<Order>
+findCustomerByOrder(o: Order): Customer
+getOldestReadyToPayCustomer(): Customer
+recordPayment(c: Customer)
+averagePaidSatisfaction(): float
+getPaidCount(): int
+getTables(): List<Table>
+getWaiter(): Waiter
+getChef(): Chef
«control» ArrivalSchedule
+customerFor(turn: int): Customer
Outcome
VICTORY
DEFEAT
boundary
1
1
«boundary» ConsoleUI
-scanner: Scanner
+showTurnStart(r: Restaurant, turn: int)
+promptWaiterAction(r: Restaurant): StaffAction
+promptChefAction(r: Restaurant): StaffAction
+showError(reason: String)
+showTurnSummary(r: Restaurant, turn: int)
+showResult(o: Outcome, r: Restaurant)

Only customer type that exists in Stage 1.
lossPerTurn()=8, discount()=no-op, chooseItem()=random.

Stage 1: only RegularCustomer arrives
(turns 1, 7, 13).

Restaurant: COMPOSITION for Tables, Staff and Menu
(all created inside its constructor and die with it).
Customers are AGGREGATED - they arrive and leave
independently of the Restaurant's own lifecycle.